

Arquitectura de Computadoras (72.08)

Manual de Usuario



Grupo 5

Integrantes:

- Fraga, Santiago - 64346
- Goldbaum, Carolina - 65112
- Choe, Ivonne - 64880

Fecha de entrega: 5 de noviembre de 2025

Requisitos previos

Para ejecutar correctamente el sistema operativo desarrollado sobre la arquitectura **x64 BareBones**, se necesita un entorno **Linux** con los siguientes componentes:

- **Docker**: requerido para la compilación dentro del contenedor.
- **QEMU**: emulador utilizado para la ejecución del sistema.

En caso de estar en Windows, se puede usar WSL para tener el entorno necesario.

Ejecución del sistema

Antes de comenzar, abrir una terminal ubicada en la carpeta raíz del proyecto.
Los scripts deben ejecutarse en el siguiente orden:

1. **./create.sh**
 - Configura el entorno de trabajo.
 - Crea un contenedor de Docker (por ejemplo, tpe_arqui_2q2025) y descarga la imagen base utilizada para compilar.
2. **./compile.sh**
 - Inicia el contenedor.
 - Elimina archivos objeto (.o) de compilaciones previas.
 - Compila nuevamente todo el código fuente.
 - Cierra el entorno al finalizar.
3. **./run.sh**
 - Ejecuta **QEMU** con los parámetros de arranque adecuados, cargando el kernel compilado.

Uso de la shell

La shell integrada permite ingresar comandos para interactuar con el sistema. En caso de ingresar un comando inexistente, el sistema mostrará un mensaje de error e indicará el uso de help para ver la lista completa de comandos.

- Escribir el nombre del comando (no distingue entre mayúsculas y minúsculas).
- Presionar ENTER para ejecutarlo.
- Durante la escritura:
 - **Backspace**: borra el último carácter.
 - **Tab**: inserta espacios (útil para alineación)

Comandos Disponibles:

Comando	Descripción	Categoría
help	Ver lista de comandos	General
time	Ver fecha y hora	General

clear	Limpiar pantalla	General
zoomin	Ampliar texto	Visualización
zoomout	Reducir texto	Visualización
tron	Jugar TRON	Entretenimiento
regs	Ver registros de CPU	Debugging
cerodiv	Generar excepción (división por cero)	Testing
invalido	Generar excepción (opcode inválido)	Testing
loop	Bucle infinito con captura de registros	Testing
cancion	Tocar música (Super Mario Bros)	Entretenimiento
exit	Cerrar shell y termina la ejecución	General

Lista de Comandos

COMANDO: **help**

Muestra todos los comandos disponibles junto con una breve descripción de su función.

```

Bienvenido a la Shell
Escribe 'help' para ver los comandos disponibles

>help
Comandos disponibles:
  help      - Muestra este mensaje de ayuda
  time      - Muestra la fecha y hora actual
  zoomin    - Aumenta el zoom del texto
  zoomout   - Disminuye el zoom del texto
  tron      - Inicia el juego TRON
  regs      - Muestra los registros guardados
  cerodiv   - Ejecuta la division por cero
  invalido  - Dispara excepcion por opcode invalido
  cancion   - Reproduce Mario Bros
  loop      - Loop infinito para testear captura de RIP
  clear     - Limpia la pantalla
  exit      - Sale de la shell
>

```

- COMANDO: **time**

Descripción: Muestra la fecha y hora actual del sistema.

Salida esperada: Fecha: DD/MM/AAAA Hora: HH:MM:SS

- COMANDO: **clear**

Descripción: Limpia la pantalla, borrando todo texto visible y dejando un fondo negro.

- COMANDO: **zoomin**

Descripción: Aumenta el tamaño del texto en pantalla (zoom de ampliación).
Nivel máximo de zoom: 5x.

```

Bienvenido a la Shell
Escribe 'help' para ver los comandos disponibles
>zoomin
Zoom in aplicado
>help
Comandos disponibles:
  help      - Muestra este mensaje de ayuda
  time      - Muestra la fecha y hora actual
  zoomin    - Aumenta el zoom del texto
  zoomout   - Disminuye el zoom del texto
  tron      - Inicia el juego TRON
  regs      - Muestra los registros guardados
  cerodiv   - Ejecuta la division por cero
  invalido  - Dispara excepcion por opcode invalido
  cancion   - Reproduce Mario Bros
  loop      - Loop infinito para testear captura de RIP
  clear     - Limpia la pantalla
  exit      - Sale de la shell
>

```

- COMANDO: **zoomout**

Descripción: Disminuye el tamaño del texto en pantalla (zoom de reducción).
Nivel mínimo de zoom: 1x (tamaño normal).

- COMANDO: **tron**

Descripción: Inicia un juego tipo *TRON* en tiempo real. Es un juego de motos de luz donde los jugadores deben evitar las líneas que dejan mientras se mueven.

Menú inicial: Elige entre 1 jugador, 2 jugadores o jugar contra la IA.

Controles:

- **Jugador 1:** Flechas de dirección o teclas **WASD**
- **Jugador 2 (modo 2 jugadores):** Teclas **WASD** y **IJKL**
- **IA:** Se mueve automáticamente (en modo vs IA)

Objetivo:

No chocar con tu propia línea, las paredes, la línea del oponente o la IA.

FPS:

El juego muestra los fotogramas por segundo en la esquina superior derecha.

Marcador:

En modo 2 jugadores, se muestra el puntaje de ambos jugadores.

- COMANDO: **regs** (Registros)

Descripción: Muestra los valores de todos los registros de la CPU guardados en el momento en que se presionó la tecla **ESC**.
 Útil para *debugging* y análisis de excepciones.

Nota: Es necesario apretar **ESC** antes de correr este comando, de lo contrario no funciona.

- COMANDO: **cerodiv** (División por Cero)

Descripción: Ejecuta una operación de división por cero para generar una excepción. Útil para probar el manejo de excepciones del sistema.

Efecto esperado: El kernel captura la excepción y muestra los registros al momento de la captura.

```
=====
EXCEPCION: DIVISION POR CERO (0x00)
=====
El sistema intento dividir por cero.

Instruction Pointer (RIP): 0x00000000040102D

Registros del procesador:
RAX: 0x000000000000002A      RBX: 0x0000000000000000
RCX: 0x0000000000000000      RDX: 0x0000000000000000
RSI: 0x00000000000403031     RDI: 0x0000000000000000
RBP: 0x0000000000010EF68     RSP: 0x0000000000010EF58
R8 : 0x000000000000FFF00     R9 : 0x0000000000000000
R10: 0x0000000000000000     R11: 0x0000000000000000
R12: 0x0000000000000000     R13: 0x0000000000000000
R14: 0x0000000000000000     R15: 0x0000000000000000

CS: 0x0000000000000008      SS: 0x0000000000000000
RFLAGS: 0x000000000000202

Presione Enter para volver a la shell...
=====
```

- COMANDO: **invalido** (Opcode Inválido)

Descripción: Ejecuta una instrucción de máquina inválida para generar una excepción de *operación inválida*.

Efecto esperado: El kernel captura la excepción y muestra los registros al momento de la captura.

```
=====
EXCEPCION: OPCODE INVALIDO (0x06)
=====
Se encontro una instruccion invalida.

Instruction Pointer (RIP): 0x000000000402E57

Registros del procesador:
RAX: 0x000000000400B4D      RBX: 0x0000000000000000
RCX: 0x0000000000000000      RDX: 0x0000000000000000
RSI: 0x00000000000403039     RDI: 0x0000000000000000
RBP: 0x0000000000010EEE0     RSP: 0x0000000000010EED0
R8 : 0x000000000000FFF00     R9 : 0x0000000000000000
R10: 0x0000000000000000     R11: 0x0000000000000000
R12: 0x0000000000000000     R13: 0x0000000000000000
R14: 0x0000000000000000     R15: 0x0000000000000000

CS: 0x0000000000000008      SS: 0x0000000000000000
RFLAGS: 0x000000000000206

Presione Enter para volver a la shell...
=====
```

- COMANDO: **loop** (Bucle Infinito)

Descripción: Inicia un bucle infinito que incrementa un contador.

Diseñado para capturar el valor de **RIP** en *userland* (instruction pointer) durante una ejecución continua.

- COMANDO: **cancion** (Reproductor de Música)

Descripción: Toca una canción (tema de *Super Mario Bros*) usando el altavoz de la PC.

Detalles: La canción se toca secuencialmente, sin interrupciones antes de que termine.

- COMANDO: **exit**

Descripción: Cierra la shell y termina la ejecución del programa.

Efecto: El sistema termina. Deberás reiniciar la máquina o la emulación.